

Online Hierarchical IRT with Assumed-Density Filtering for Predictive AI Evaluation

Haoyuan Luo — University of California, Davis — zixluo@ucdavis.edu

Technical report — Predictive AI Evaluation Competition, NeurIPS 2026. Codabench submission **955485** (paiec_irt.zip, sha256 98555787...2cfe0f). Code: <https://github.com/lucaluo925/paiec-2026>

Abstract

A decision maker choosing between AI systems can rarely afford to evaluate them on every item. This competition asks how well a system’s per-item success can be predicted on a benchmark never seen during development, given a budget of 0 to 31 labels for the subject-benchmark pair under test. We model the response as a hierarchical item-response model — $\text{logit } P(\text{correct}) = \alpha - \theta_s + v_s - \beta - d_i - \sum w_f$ — with subject ability θ_s calibrated offline on the public measurement-db and predicted from provider, release date, parameter count and name tokens for subjects absent from calibration. At evaluation time we hold one joint Gaussian posterior per anonymous benchmark and absorb each acquired label with an assumed-density-filtering step (probit approximation, closed-form rank-one update). Labels of *other* subjects on the same benchmark inform the shared latents, which is where most of the gain at small budgets comes from.

Under leave-one-benchmark-out validation on four public benchmarks (147 subject-benchmark pairs, 19,621 evaluation targets), the fully out-of-fold model reaches Brier ALC **0.16875** against **0.23110** for the organizers’ empirical-mean baseline and 0.25000 for a constant 0.5; cross-fitting the shrinkage weights under the same protocol gives an honest **0.16515**. The advantage does not depend on having met the model before: on the 27 pairs whose identity calibration never saw, ALC is 0.18424 against 0.24960.

Two negative results are reported as fully as the positive one. First, the competition’s formative feedback is far noisier than it appears: nine byte-identical submissions of the same artefact span 0.1685 to 0.2062 — a standard deviation of **0.011** on ALC and 0.011–0.023 per budget, a range of 3.3 SD between two evaluations of the same file, because each evaluation redraws only eight or nine subject-benchmark pairs. Of everything that feedback could be asked, three statements survive it: (i) the gap to the leading entry; (ii) B_0 above our local estimate; and (iii) B_{31} above our local estimate. We state what does not survive it as well. Second, four pre-registered mechanism-level candidates were falsified on leave-one-benchmark-out — three aimed at the zero-label prior, and a fourth, text-residual propagation inside the evaluated benchmark, which cleared one of its four pre-set gates and improved pooled ALC by 0.0005 against a required 0.045. The shipped configuration was left unchanged. We also record a sequencing error of our own: the noise calibration was run after, not before, the first live score had been used to draw conclusions.

Keywords: predictive evaluation; item response theory; assumed density filtering; online Bayesian updating; calibration; label-efficient evaluation; Brier score.

Scope of the numbers. Every figure outside §5b is a local hold-out estimate on the public measurement-db, produced by the scripts in this repository. §5b reports the live Codabench scores and treats them as a repeated measurement of noise, not as a selection signal; the leaderboard was never consulted for model selection.

1. Task and scoring

A predictor receives a subject (eight visible attributes: `normalized_name`, `provider`, `release_date`, `access_date`, `harness`, `reasoning_effort`, `harness_version`, `subject_features_extra`),

an item (item_content, item_features, interactors, anonymous benchmark_id) and the labels acquired so far, and outputs $P(\text{correct})$. Brier scores at label budgets $b \in \{0, 1, 3, 7, 15, 31\}$ are combined as $\text{ALC} = 0.1 \cdot B_0 + 0.2 \cdot B_1 + 0.2 \cdot B_3 + 0.2 \cdot B_7 + 0.2 \cdot B_{15} + 0.1 \cdot B_{31}$, so 80 % of the weight lies at 1-15 labels. The official rules average Brier within each subject-benchmark pair first and then across pairs with equal weight, “regardless of its number of responses”; all numbers in this report use that pair-macro aggregation. Test benchmarks are private and unseen during development.

2. Data

- **Training corpus.** The public aims-foundations/measurement-db (Hugging Face, dataset revision bc8204d811823da849c6686bf124d4ca9f82e4de), restricted — like the organizers’ preparation script — to benchmarks with response_type = binary and granularity = item.
- **What that leaves.** The repository holds 10 benchmark directories (response_type: 6 binary, 2 mixed, 2 fraction). Two of the six “binary” directories are reproduction_checks/.../tables smoke artifacts carrying 1 and 2 responses and re-using their parent’s benchmark_id; they are not independent benchmarks and are excluded (they would also fail the ≥ 80 -item rule). **Four usable benchmarks remain**, and neither of the two excluded top-level benchmarks (matharena, mmdocrag) is replaced, so the training pool contains no mathematics and no document-understanding benchmark.

benchmark	domain	subjects	items	responses	density	base rate
multi_swebench	software engineering, agents	82	2 126	57 808	0.332	0.148
researchcodebench	software engineering, ML	31	212	6 572	1.000	0.352
swe_rebench	software engineering, agents	1	6 306	6 306	1.000	0.475
real_webagents	agents and tool use	33	233	3 759	0.489	0.375

In total 74 445 responses, 8 877 items, 147 subject configurations and 60 distinct model identities. 17 identities appear in ≥ 2 benchmarks and **none in ≥ 5** , so the cross-benchmark ability calibration rests on 17 models.

- **Attribute coverage.** normalized_name and provider 100 %, release_date 93.9 %, access_date and harness 55.8 %, harness_version 0.7 %, **reasoning_effort and subject_features_extra 0 %**. The configuration-effect terms for reasoning effort therefore have no support in this corpus. item_features is non-empty for 2 571 items (most common keys lang, website, paper); interactors is empty everywhere.
- **Preprocessing.** One response per (subject, item, interactors): the lowest (test condition, trial) row with a 0/1 grade, because the visible input cannot distinguish trials or conditions. Missing attributes become "".
- **No other data.** No external data, benchmark results, model cards or web sources were used. The model registry used for normalized names is the one already embedded in measurement-db subject tables.

3. Local evaluation protocol

All model selection used a local replica of the evaluator (`eval/local_scorer.py`), never the Codabench leaderboard.

- **Streaming protocol.** Re-implements `streaming_alc_v1` from the organizers’ `tools/streaming_ingestion.py`: benchmarks with ≥ 80 items; a 50/50 acquisition/evaluation item split **per subject-benchmark pair**; each pair’s acquisition items are streamed one at a time with `max_labels = 31`; the default policy queries when a SHA-256 uniform falls below `labels_remaining / items_remaining`; all pairs reach budget `b` before the checkpoint at `b`; `labeled` contains the labels of all sampled pairs.
- **Checked against the official rules page** (aimslab.stanford.edu/competition): the per-pair 50/50 split, the pair-macro averaging, the ALC weights, the budgets, `max_labels = 31`, the context fields (`subject_id`, `benchmark_id`, `labels_acquired`, `labels_remaining`, `max_labels`, `items_remaining` including the current candidate), and the global scope of `labeled` all match. Two of our earlier assumptions did **not** match and were corrected: the split had been made once per benchmark and shared across subjects, and Brier had been averaged over predictions rather than over pairs. Both corrections are locked by regression tests (`test_official_protocol_defaults`, `test_pair_macro_weights_pairs_equally`).
- **Verification.** (i) A constant 0.5 predictor scores exactly 0.25 at every budget and in ALC (weights are integers in tenths and summed with `math.fsum`).
(ii) The organizers’ own streaming client (`run_streaming`) driven by our coordinator re-produces our acquisitions and predictions exactly (unit test).
(iii) Label sets are nested across budgets.
- **Hold-out.** Two-way: benchmarks are split into 5 folds (benchmarks sharing $\geq 5\%$ item texts are kept together), and 20 % of model identities are removed from every training fold and appear only as unseen subjects. Items are never split at random. With four benchmarks the 5-fold split degenerates to **leave-one-benchmark-out** (fold sizes 1, 1, 1, 1, 0). Local evaluation uses every subject configuration in the corpus: 147 pairs and 19 621 evaluation targets, of which 27 pairs / 3 556 targets belong to the 12 held-out identities.
- **Still unverified.** The rules page does not state the evaluation worker concurrency or the size of the private test set. The engine holds an `RLock` and is safe under threads; process-level parallelism only re-creates state. The caps we pass locally (≤ 200 subjects per benchmark, ≤ 200 evaluation items per pair, ≤ 800 streamed items per pair) are ours, not the rules’; at these values none of them binds on this corpus, so the numbers in §5 use the whole public dataset. §5b’s local column is a smaller configuration, at the harness’s default caps, and says so where it is used. The rules’ own cap — 1 000 unique subject-item pairs — applies to formative feedback runs only.

4. Method

4.1 Model

For benchmark B , subject s and item i ,

$$\text{logit } P(y=1) = a \cdot \theta_s + v_s - \beta - d_i - \sum_f w_f$$

- θ_s — general ability, **calibrated offline**. Known models use their fitted ability; unseen models use a ridge regression on provider, release date, parameter count (parsed from the name) and name tokens, plus learned effects of reasoning effort / harness settings.
- a, β — the benchmark’s discrimination of general ability and its difficulty; priors are the across-benchmark distribution of item-level fits.
- $v_s \sim N(0, \tau^2 + (\bar{a}^2 + \text{var } a) \cdot \text{var } \theta_s)$ — subject $\times$ benchmark deviation.

- $d_i \sim N(0, \sigma_d^2)$ — item difficulty; $w_f \sim N(0, \sigma_w^2)$ — effects of the item’s `item_features` / `interactors` tokens, learned within the benchmark.

4.2 Offline calibration (`train/fit_offline.py`)

1. Subject abilities by weighted alternating least squares on per-(benchmark, subject) logit accuracies: $\text{logit}(\text{acc}) = a_B \theta_u - b_B$, with θ of a configuration unit shrunk towards its model’s ability plus configuration effects.
2. Item side: for every training benchmark, an item-level model (a, β, v_s, d_i with variance components τ^2, σ_d^2) by block-coordinate Newton with EM variance updates. The distribution of these item-side parameters across benchmarks gives the priors for a new benchmark. Item-level parameters are never transferred directly, because the test benchmarks are new.
3. Hyper-parameter scales and the shrinkage weights are selected on the local folds (`eval/tune.py`), with a cross-fitted estimate for honesty. Selected scales: `delta_var` 1.00, `tau2` 0.21, `beta_var` 6.76, `a_var` 0.60, `w_var` 4.16. The coordinate search over the scales was run on a 40-subject-per-benchmark subsample, because a search under the full protocol costs about 20 s per candidate and does not fit the compute available here; the shrinkage weights are then fitted **and cross-fitted under the full protocol** (`--scales ... --rounds 0 --crossfit`). This is disclosed because the scales are therefore selected on a smaller sample than the one they are reported on.

4.3 Online update (`submission/pirt_online.py`)

One joint Gaussian over all latents of a benchmark. Each label is absorbed by an assumed-density-filtering step: the logistic likelihood is approximated by a probit, whose moments are closed-form, followed by a rank-one covariance update. Labels are processed one at a time in the order supplied; the state is cached and extended incrementally when the evaluator’s label list grows. Labels of other subjects on the same benchmark update the shared latents (β, a, d_i, w_f). A numpy-free scalar fallback runs if numpy is unavailable.

4.4 Prediction and safety net

$p = E[\sigma(\text{logit})]$ under the posterior (probit approximation), then $p' = (1 - w_n) \cdot p + w_n \cdot \bar{p}$ with $\bar{p}$ the training base rate and w_n chosen per label count on local validation. Selected weights: $w_0 = 0.535$, $w_1 = 0.141$, $w_3 = 0.000$, $w_7 = 0.015$, $w_{15} = 0.004$, $w_{31} = 0.003$; final $\bar{p} = 0.2816$.

w_n is the weighted least-squares optimum in which each prediction of a pair with n targets carries weight $1/n$, matching the way the metric averages; pooling the predictions instead would let the largest pairs choose w for every pair. On this corpus the two agree to within $1e-5$ of ALC, so the alignment is a correctness property rather than a source of gain.

$\bar{p}$ is the mean over training subject-benchmark pairs of each pair’s accuracy, not the pooled response rate. This matters: pooling lets the largest benchmark set the target, which here would give 0.21 against a pair-weighted 0.31, and cost about 0.003 of B_0 .

4.5 Acquisition (`labeling.py`) — not shipped

Streaming decision per candidate. The value of a candidate is the expected reduction of the posterior variance of the pair ability $c = a \theta_s + v_s - \beta$ (Fisher information under the current posterior, including the candidate’s difficulty uncertainty), relative to a generic unseen item: ρ . The query probability is $\min(1, q \cdot \rho^2)$ with $q = \text{labels_remaining} / \text{items_remaining}$; with $\rho \equiv 1$ it is exactly the evaluator’s random policy.

A single pre-registered A/B (eval/ab.py, results/acquisition_ab.md) compared the random policy against Fisher information and against posterior uncertainty, with the decision rule fixed beforehand: ship only if ALC is lower **and** the 95 % interval of a paired bootstrap over benchmarks excludes 0.

policy	ALC	Δ ALC vs random	95 % CI (benchmark clusters)
random (official)	0.16875	—	—
Fisher information	0.16981	+0.00106	[−0.00071, +0.00345]
posterior uncertainty	0.16846	−0.00029	[−0.00321, +0.00448]

Neither interval excludes 0, so **the submission ships without labeling.py** and uses the official random policy. The A/B was first run on the 40-subject subsample (Fisher +0.00127, uncertainty +0.00156, both intervals straddling 0) and then re-run unchanged on the full protocol, which is the table above; the decision is the same under both, and the rule was not altered after seeing either. Posterior uncertainty turns marginally negative on the larger sample but its interval is four times wider than the effect. Fisher information is better at B₁ and B₃₁ and worse at B₃–B₁₅, where the ALC weight is concentrated; we report this as an observation, not as grounds for a third attempt.

5. Results (local hold-out)

Cross-validated, leave-one-benchmark-out, pair-macro Brier; lower is better; 147 pairs and 19 621 evaluation targets. The rows below are the fully out-of-fold model **before** hyperparameter tuning, so no selection of any kind enters them.

Predictor	ALC	B ₀	B ₁	B ₃	B ₇	B ₁₅	B ₃₁
Constant	0.25000	0.2500	0.2500	0.2500	0.2500	0.2500	0.2500
0.5							
Empirical mean (organizers)	0.23110	0.2500	0.3449	0.2310	0.1902	0.1774	0.1740
Online IRT	0.16875	0.2211	0.1932	0.1771	0.1593	0.1412	0.1248
Online IRT, own-pair labels only	0.19450	0.2211	0.2117	0.1985	0.1875	0.1777	0.1731

That is 27 % better than the organizers’ baseline on ALC. With the shrinkage weights fitted and cross-fitted under the same protocol, the **cross-fitted (honest) estimate** is **ALC 0.16515**, per-budget Brier 0.2061 / 0.1897 / 0.1724 / 0.1574 / 0.1408 / 0.1248.

Unseen model identities (27 of 147 pairs, calibration never saw the identity): Online IRT 0.18424, empirical mean 0.24960, constant 0.5 0.25000. Seen identities: 0.16526 / 0.22693 / 0.25000. The advantage does not depend on having met the model before.

Per fold the ALC is 0.1509, 0.1943, 0.2728 and 0.1860; the worst fold is swe_rebench, which contributes exactly one pair.

These four numbers do not average to the headline. The headline is a mean over pairs, and the four folds hold 82, 31, 1 and 33 of the 147 pairs: $(82 \cdot 0.1509 + 31 \cdot 0.1943 + 1 \cdot 0.2728 + 33 \cdot 0.1860) / 147 = 0.16876$, which is 0.16875 up to the rounding of the per-fold values. The unweighted mean of the four folds is 0.201, and the spread of these four points, unweighted, is a standard error of 0.026. §5b and §6 use the same unweighted statistic, but computed on the folds of the configuration §5b compares against (0.041; see §5b, reading 1). The two aggregations answer different questions (how well the model does per pair, and how much that changes from one benchmark to the next) and should not be compared with each other.

Calibration improves with labels: expected calibration error $0.122 \rightarrow 0.072 \rightarrow 0.058 \rightarrow 0.023 \rightarrow 0.027 \rightarrow 0.016$ across the six budgets (figures/calibration.pdf). At zero labels the predictor is visibly over-confident; this is what the shrinkage weight $w_0 = 0.535$ absorbs.

Figures: figures/learning_curves.pdf, figures/calibration.pdf; acquisition A/B: results/acquisition_ab.md; data survey: results/data_survey.md; code review notes: results/ocr_review*.md.

5b. Results (live, Codabench formative feedback)

The shipped artefact (dist/paiec_irt.zip, sha256 98555787...2cfe0f, 15,436 bytes) was submitted **nine times unchanged** between 2026-10-01 and 2026-10-03. Every one of the nine was verified to be the same file: the uploaded bytes were re-downloaded from the platform and hashed, and all nine give the same sha256. The rules state that “*each formative evaluation uses a newly sampled subset of the hidden test data, capped at 1,000 unique subject-item pairs*”, so the nine runs differ only in which subject-benchmark pairs were drawn. We report them as a repeated measurement, because a single live score turns out to be far noisier than we expected, and that is the main thing they taught us.

The nine Brier ALCs, in submission order: 0.20624, 0.20098, 0.18124, 0.18685, 0.19461, 0.18850, 0.18203, 0.18458, 0.16850. Each run drew eight or nine subject-benchmark pairs, and in every one the unweighted mean of the per-pair ALCs reproduces the reported total to six decimals — nine independent confirmations that the official aggregation is an equal-weight mean over pairs.

	mean	SD, one draw	SE of mean	local	gap	runs above local
Brier ALC	0.18817	0.0113	0.0038	0.17540	+0.0128	8 / 9
B ₀	0.27757	0.0224	0.0075	0.21634	+0.0612	9 / 9
B ₁	0.21707	0.0235	0.0078	0.20171	+0.0154	6 / 9
B ₃	0.18521	0.0159	0.0053	0.18436	+0.0008	5 / 9
B ₇	0.16484	0.0112	0.0037	0.16570	−0.0009	5 / 9
B ₁₅	0.15746	0.0125	0.0042	0.15042	+0.0070	7 / 9
B ₃₁	0.15500	0.0107	0.0036	0.13330	+0.0217	9 / 9

The local column is not the \$5 configuration. It is the shipped configuration (tuned hyper_scale and shrinkage weights from results/tuned_hyper_full.json) under leave-one-benchmark-out with the harness’s default sampling caps (--max-subjects 40 --max-eval 60 --max-stream 400): 105 pairs and 5,964 targets, global label scope, per-pair split. It is reproduced to the printed digits by `python -m eval.run_cv --data data/raw --variants irt --tuned results/tuned_hyper_full.json`.

The measurement noise is the headline result. A single formative score has a standard deviation of about **0.011** on ALC and 0.011–0.023 per budget. The nine runs span 0.1685 to 0.2062, a range of 0.0377 — **3.3 SD between two evaluations of the same bytes**. That is what one formative score is worth, and it is the number a participant reading their own leaderboard entry needs.

Two SDs, and they answer different questions. The 0.011 above is the spread of a *single* draw. The precision of the *mean of nine* is $0.011/\sqrt{9} = \mathbf{0.0038}$, and that is the right denominator for comparing this entry against anything. An earlier version of this section used the single-draw SD for those comparisons and so understated every one of them; the readings below are restated. Where the other side of a comparison is itself a single formative draw — another team’s published score — the combined scale is $\sqrt{(0.0038^2 + 0.0113^2)} = 0.0119$.

1. **We cannot resolve the live-versus-local difference, and the live side is not what limits us.** The gap is +0.0128. Against the live mean’s own SE that would be 3.4 SD,

but the local figure is not a constant: it comes from leave-one-benchmark-out over four benchmarks whose per-fold ALCs, in the configuration of the local column, are 0.1556, 0.1867, 0.3361 and 0.1839, a standard error of **0.041** — about eleven times the live one. Carrying both gives $0.0128 / 0.041 = \mathbf{0.3\ SD}$: no resolvable difference, and the binding uncertainty is our own four-benchmark estimate, not the competition’s sampling. Eight of nine draws land above the local value, which is a weak signal on its own (sign test $p = 0.04$).

2. **Two budgets separate cleanly from the local estimate, and they are the two we expected least.** B_0 and B_{31} are each above local in **all nine** draws (sign test $p = 0.004$ two-sided, which does not depend on the local standard error at all), at $+0.061$ and $+0.022$. B_1 does not: 6 of 9, $p = 0.51$. An earlier version of this section said the shortfall was concentrated in “the two lowest budgets” and then that it was at B_0 alone; with nine draws and a test that does not lean on the local SE, it is **B_0 and B_{31}** — the two end points, not the low end. B_3 and B_7 are at 5 of 9, $p = 1.0$.
3. **We are behind the leading entry and that is the one competitive fact the feedback establishes.** The best public entry scores 0.117238. That is itself a single formative draw, so the comparison carries both sides: $+0.071$ against a combined 0.0119, **6.0 SD**, and not one of our nine draws came close. Against the organizers’ own entry at 0.180113 the gap is $+0.008$, **0.7 SD**, with one of nine below it — we cannot claim to be better or worse than their baseline. The 27 % margin over the empirical-mean baseline in \$5 is a statement about that baseline method on local hold-out data and should not be read as competitiveness.

One caution about that leading score, which follows from the same argument this section makes about our own. A Brier of 0.117238 is exactly $\mu(1-\mu)$ at $\mu = 0.1356$, so it is also what a predictor with **no discriminative power at all** scores on a pool whose base rate happens to be 0.136 — the pool-wide rate across the four public benchmarks is 0.338, where that same zero-skill predictor would score 0.224. A single formative score therefore cannot separate a strong predictor from an easy draw, and we do not know the base rate of the pairs drawn for that submission. What our own nine draws do establish is the scale of draw-to-draw movement for one fixed predictor: 0.0113, with a minimum of 0.1685. On that scale 0.117238 sits 6.3 single-draw SDs below our mean, so we are not going to explain it as luck — but neither should it be read as a measured algorithmic advantage on a common sample. The summative evaluation uses a common hidden subset, and that is the first number from this competition that will support a comparison between teams.

4. **$\hat{\sigma}$ is not monotone in n , which is the reason we did not stop at a convenient value.** It ran 0.0132 ($n=3$), 0.0117, 0.0102, 0.0094, 0.0095, 0.0091, 0.0113 ($n=9$): it dipped below the 0.01 threshold we had pre-registered for discounting single-score inferences and came back above it when the ninth draw landed at 0.1685. Had we stopped at $n=6$ the pre-registered discount would have lifted on an artefact of sampling. The threshold is met at $n=9$, but the lesson is the shape of that sequence, not which side of 0.01 it finishes on.
5. **Platform status is not a reliable filter.** Two of the nine were marked Failed after producing complete, internally consistent scoring outputs, and of the fourteen submissions of this artefact one has sat in Submitting for over two days and two more produced nothing in eleven hours. These two Failed runs are among the nine that scored; they are not the two that failed outright, which are counted separately at the end of this section. Restricting the pool to the seven the platform calls Finished moves the single-draw SD to 0.0130 and leaves every conclusion above intact, B_0 and B_{31} still above local in all seven.

We pre-registered and tested three mechanism-level fixes for the zero-label prior: shrinking

towards 0.5, estimating the target benchmark’s success rate online with a 0.5 cold start, and the same with the original constant as the cold start. Gates were fixed before running; **all three failed** on leave-one-benchmark-out, each making B_0 and/or B_1 worse. The constant was fitted on exactly the four public benchmarks, so for them it is close to the best available prior rather than a wrong one. The live shortfall at B_0 — now the one per-budget effect that clears the noise — appears to involve a benchmark whose success rate lies outside the 0.148–0.475 range the public pool spans, which no local protocol can reproduce, so we record the hypothesis as **untested rather than refuted**. The shipped configuration is unchanged.

A fourth candidate was pre-registered and run after those three, because an external review argued it was the one mechanism our earlier findings did not already exclude. **Text-residual propagation**: inside an unseen benchmark, use the text of `item_content` to relate a target item to the items already labelled, and carry their baseline residuals across. The point is that it needs no difficulty coefficient to transfer between benchmarks — the transfer that §4.2 shows does not work — only a within-benchmark relation between text similarity and residual. Implementation was constrained to the submission environment: fixed word and character n-gram hashing, a fixed kernel, a zero-mean correction on the logit, a single kernel strength fitted in closed form on the training benchmarks with no validation search, and a correction that is **strictly zero** when the evaluated benchmark has no labels yet.

Four gates were fixed before running, in `docs/PREREG_text_residual.md`, which was written before any code for this candidate was run: $\Delta\text{ALC} \geq 0.045$; all three multi-pair folds improve; the single-pair fold degrades by at most 0.005; and a control that shuffles the text identity of the support items moves ALC by at most 0.002. Cross-subject evidence was replayed at the cohort size the live runs actually expose — eight or nine pairs — rather than handing the predictor every subject in the public benchmark, which would inflate the gain.

fold	baseline	text residual	shuffled control	ΔALC	λ
0 (9 pairs)	0.16012	0.15868	0.16009	+0.00144	0.392
1 (9 pairs)	0.21105	0.20972	0.21105	+0.00133	0.422
2 (1 pair, <code>swe_rebench</code>)	0.32194	0.33239	0.32227	−0.01045	0.370
3 (9 pairs)	0.18481	0.18481	0.18481	0.00000	0.000
pooled	0.19021	0.18969	0.19021	+0.00052	—

One gate of four. Only the control passed, at exactly 0.00000. ΔALC is +0.00052 against a required 0.045 — 1.2 % of the threshold; fold 3 did not improve; and the single-pair fold degraded by 0.0105, twice its allowance. The mechanism is refuted and the shipped configuration is unchanged.

The failure is informative in three ways. **The effect is real and simply far too small**: shuffling the text identity moves the pooled ALC by 0.00000 while the true text moves it by +0.00052, so that half-thousandth is a text relation rather than noise. It is two orders of magnitude short of what a different entry would need. **On fold 3 the fitted λ was exactly zero**: the three training benchmarks for that fold supported no usable text-to-residual relation, and the method declined to act, which is the behaviour we wanted but also shows the mechanism does not hold across benchmarks. **The harm concentrated on the one-pair fold**, the shape the heterogeneity analysis warned about — a method that helps most benchmarks and hurts one.

One implementation note that cost us a wrong reading before we caught it. We briefly narrowed the hash width from 4096 to 512 to save computation, and the λ fitted on the training benchmarks fell from 0.392 to 0.054. **The width of the text representation is not a neutral implementation constant**; a narrow hash erases the signal being measured. We restored 4096 and bought the computation back by shrinking the replayed cohort instead — which the pre-registration required anyway.

No model choice in this report was made using live feedback. Formative evaluation resamples on every submission and the summative evaluation uses a separate common subset, so selecting on it would be selecting on noise. All nine scored runs above were spent on measuring that noise, not on choosing anything. Of the fourteen submissions of this artefact, nine produced a score, two failed outright on the platform and three have remained queued without producing output (one for over two days); we report the nine that scored. The two that failed outright produced no score at all, and are a different pair from the two scored runs marked Failed in reading 5.

6. Limitations

- **The training pool is very small and narrow.** Four benchmarks, all software engineering / agent tasks; no mathematics, no document understanding. The private test benchmarks may well include domains with no representative here, and the across-benchmark priors that the method depends on are estimated from four points.
- **The local estimate is coarse.** Leave-one-benchmark-out over four benchmarks gives per-fold ALC of 0.151, 0.194, 0.273 and 0.186. Any difference smaller than that spread — including the 0.004 gained by hyper-parameter tuning — is not resolvable with this data. `swe_rebench` contributes a single subject-benchmark pair, and on that fold the model (0.273) is worse than a constant 0.5; a benchmark with one subject gives the offline calibration nothing to learn from.
- **The live estimate is coarse too, and for a different reason.** Each formative evaluation draws eight or nine subject-benchmark pairs afresh, so a single live score carries a standard deviation of about 0.011 on ALC and 0.011–0.023 per budget (§5b, nine identical submissions). The mean of nine is far tighter, SE 0.0038, so the live side is not what limits the comparisons; our own leave-one-benchmark-out estimate is, with a four-fold standard error of 0.041. The live feedback supports three statements: (i) the gap to the leading entry is real (6.0 SD once that entry is also treated as a single draw); (ii) B_0 exceeds our local estimate in all nine draws; and (iii) so does B_{31} (sign test $p = 0.004$ for each, which does not depend on the local standard error). Nothing else is resolved — not the overall live-versus-local gap (0.3 SD once both uncertainties are carried), not whether we are behind the organizers’ own entry (0.7 SD), and not any other per-budget reading. We measured this only after drawing conclusions from the first live score, which was a mistake in sequencing, not just in arithmetic.
- **The pre-registered bootstrap is weak by construction.** Resampling four benchmark clusters yields a coarse distribution. We kept the rule rather than changing it after seeing the data, and report a pair-clustered interval alongside as a diagnostic; both agree that the acquisition variants are not distinguishable from random.
- **Zero-label prediction is the weakest budget, but it does beat a constant.** B_0 is 0.2061 cross-fitted. The right comparison is a constant chosen the same way the model is fitted — the base rate of the training folds only — which scores 0.2122; the model is 0.006 better. An earlier draft of this report claimed the opposite by comparing against the constant that minimises error on the evaluation pool itself (0.2160 at the smaller protocol), which is not a baseline any predictor could have chosen in advance. This is a statement about local hold-out data against a base-rate constant, and it does not carry over to the live setting: §5b finds live B_0 above the constant-0.5 value of 0.2500 in all nine runs, and above our own local B_0 in all nine ($p = 0.004$). The zero-label prior is the one place where local and live disagree beyond the noise. The weighted least squares for w_n described in §4.4 was aligned with the metric after that finding and changed the result by less than $1e-5$ of ALC, confirming that the earlier gap was an artefact of the unfair baseline, not a weighting bug.
- **Within-benchmark text propagation was tried and does not close the gap.** §5b’s fourth candidate measured it directly: a real but 0.0005-sized effect on pooled ALC, against the 0.045 that would have justified a new submission. We cannot rule out that a

stronger text representation would do better — ours was constrained to hashed n-grams by the no-network submission environment, and the one time we narrowed that representation the measured signal fell by a factor of seven, which suggests the representation matters more than the mechanism. What we can say is that the version runnable inside this competition’s constraints does not.

- **reasoning_effort has no support.** The configuration-effect machinery for it is inert on this corpus; if the private subjects carry that field, its effect is untrained.
- Items of the evaluation pool are never labeled, so item-level variation comes only from visible features; the text of items is not used by the model beyond feature tokens. The text-difficulty prior (Task 4 of our plan) was not built.
- One latent ability dimension plus benchmark-specific deviations; skills that do not correlate with general ability are captured only through `v_s`.
- ADF is order-dependent and approximates the posterior.
- Evaluation worker concurrency is unstated in the rules; the engine is thread-safe but has not been tested under the organizers’ actual harness.

7. Reproducibility

```
python3.11 -m venv .venv && . .venv/bin/activate
pip install -r requirements-lock.txt
git clone --depth 1 https://github.com/aims-foundations/paiec_baseline # pinned at 82d330d
export PAIEC_BASELINE_DIR=<path to paiec_baseline>
export HF_TOKEN=<read token; the dataset is gated>
CAPS="--max-subjects 200 --max-eval 200 --max-stream 800"
python scripts/download_measurement_db.py --out data/raw
python scripts/survey_data.py --data data/raw --out results/data_survey.md
python -m eval.run_cv --data data/raw --variants const0.5 empirical_mean irt irt_pairscope \
    $CAPS --out results/by_budget.csv --records results/cv_records.npz
python -m eval.tune --data data/raw --crossfit --out results/tuned_hyper.json # scale search, 40-
subject subsample
python -m eval.tune --data data/raw --crossfit --rounds 0 --scales results/tuned_hyper.json \
    $CAPS --out results/tuned_hyper_full.json # shrinkage under the full protocol
python -m eval.ab --data data/raw $CAPS --out results/acquisition_ab.md
python tools/make_figures.py --records results/cv_records.npz --curves irt empirical_mean irt_pairscope \
    --calibrate irt --ab "irt:random" "irt+uncertainty:Posterior uncertainty" --ab-records results/ab_records.npz
python -m train.train_final --data data/raw --tuned results/tuned_hyper_full.json
python tools/build_submission_zip.py
python $PAIEC_BASELINE_DIR/check_submission_zip.py dist/paiec_irt.zip
```

Seeds are fixed (split seed 0, deterministic SHA-256 hashing). Verified on Python 3.11.16 with the locked dependency versions: two `run_cv` runs produce identical CSVs, two `train_final` runs produce identical `params.json`, and two ZIP builds are byte-identical to each other. The organizers’ `check_submission_zip.py` passes on the shipped artefact without `--static-only` (re-checked before release).

The rebuilt ZIP does not have the same hash as the scored one, and this is expected.

The artefact that was scored, and the artefact that `tools/build_submission_zip.py` produces from the source as published, are:

```
scored sha256 98555787478009fc9e53b05a7a532912db37b716e24ed9abac121bac33c2fe0f
rebuilt sha256 21a9df6f4323d94828280e8d6514c0579bb92b0f87b2ddc52371079279743a8a
```

Four of the five packaged files are byte-identical; the difference is confined to `pirt_online.py`, which gained an inert `shrink_target == "bench"` branch while the three zero-label candidates of §5b were being tested after the artefact was built. The shipped `params.json` carries no `target` key, so the default `"const"` path runs and the two versions are not merely equivalent in principle: over 300 randomized (subject, item, label-sequence) triples with 0 to 31 labels they return bit-identical predictions, with a maximum absolute difference of 0. We kept the falsified candidates in the source rather than stripping them, because a candidate that was pre-registered and refuted is a result, and deleting it would make the record of §5b unverifi-

able. Extracted into an empty directory and imported in a fresh process, the model predicts in single-digit microseconds per call (6.7 μ s with numpy, 2.8 μ s on the numpy-free fallback), which is negligible against the 8-hour run limit; malformed labels are skipped and an empty label list returns the prior.

The submission ZIP contains `model.py`, `pirt_online.py`, `params.json`, `requirements.txt` and `README.md`. No `models.txt` is needed: the submission declares no Hugging Face model. The build script scans every packaged file for credentials and refuses parameters fitted on synthetic data.

8. Disclosures

- **Pretrained models:** none. The submission contains no neural network, embedding model or LLM, and makes no API calls. Nothing in the evaluation path touches the network.
- **Data:** only the public (access-gated) `aims-foundations/measurement-db` at revision `bc8204d811823da849c6686bf124d4ca9f82e4de`; see §2.
- **LLM coding assistant:** Claude Opus 5 (Anthropic), used through the Claude desktop app’s agentic coding mode, to write the code and draft this report under the author’s direction; all results were produced by the scripts in this repository.

9. Artefacts and references

Every external source this report relies on, with the version it was read at. No other source was consulted.

1. Predictive AI Evaluation Competition, NeurIPS 2026 — task definition, scoring rules and formative-evaluation policy. <https://aimslab.stanford.edu/competition> (rules page, read 2026-10-03).
2. `aims-foundations/measurement-db`, Hugging Face dataset, revision `bc8204d811823da849c6686bf124d4c` — the only training data used (§2).
3. `aims-foundations/paiec_baseline`, pinned at commit `82d330d` — the organizers’ preparation script, streaming client (`tools/streaming_ingestion.py`, `streaming_alc_v1`), empirical-mean baseline and `check_submission_zip.py`, all of which §3 reproduces or calls.
4. This submission’s code and results: <https://github.com/lucaluo925/paiec-2026>. Within it: the pre-registration of the fourth candidate (`docs/PREREG_text_residual.md`), the acquisition A/B (`results/acquisition_ab.md`), the data survey (`results/data_survey.md`), the code-review notes (`results/ocr_review*.md`) and the figures (`figures/learning_curves.pdf`, `figures/calibration.pdf`).
5. Codabench submission 955485, artefact `paiec_irt.zip` — the scored artefact behind every number in §5b; its sha256 is the first of the two printed in §7.